

Sistemas distribuidos y Paralelos

Paralelización del problema de las N reinas

Informe final

Grupo 29

Rodriguez Aguiar, Matías - 02921/9

Índice

1- Descripción del algoritmo secuencial.....	1
1.1 Representación del tablero y las Reinas.....	1
1.2 Estrategia para determinar tableros válidos e inválidos.....	1
1.3 Almacenamiento de resultados y tableros válidos.....	2
2- Estrategia y descripción de las etapas de diseño paralelo.....	3
2.1 Descomposición.....	3
2.2 Comunicación.....	3
2.3 Aglomeración.....	4
2.4 Mapeo.....	4
3- Tiempos de ejecución, métricas y análisis de escalabilidad.....	5
3.1 Tiempos de ejecución.....	5
3.2 SpeedUp relativo.....	6
3.3 SpeedUp absoluto.....	6
3.4 Eficiencia.....	7

3.5 Balance de carga.....8

3.6 Análisis de escalabilidad..... 9

1- Descripción del algoritmo secuencial

El problema de las N-Reinas consiste en colocar N reinas en un tablero de ajedrez de $N \times N$ de manera que ninguna de ellas pueda atacar a otra. Esto implica que no puede haber dos reinas compartiendo la misma fila, columna o diagonal.

1.1 Representación del tablero y las Reinas

En el algoritmo de Takaken el tablero de $N \times N$ se representa mediante un arreglo de enteros `BOARD[N]`, donde cada posición `BOARD[y]` corresponde a la fila 'y' del tablero. El valor almacenado no es un entero ordinario sino un bitmask: el bit en la posición `c` indica que la reina de la fila 'y' ocupa la columna `c`. Esta representación permite manipular posiciones y amenazas mediante operaciones de bits muy eficientes. Por ejemplo, para $N=8$, el valor `BOARD[2] = 0b00000100` indica que la reina de la fila 2 está en la columna 2 (contando desde la derecha). El bit más alto representa la columna $N-1$, y el bit más bajo representa la columna 0.

Esto genera que el algoritmo resulte muy eficiente en cuanto a cantidad de datos a manejar durante la ejecución ya que en lugar de mover una matriz de tamaño $N \times N$, solo necesitamos un vector de N enteros y otros parámetros que en conjunto resultan considerablemente menores en tamaño respecto a la matriz.

1.2 Estrategia para determinar tableros válidos e inválidos

El algoritmo emplea backtracking con poda temprana combinado con la explotación de simetrías del tablero para reducir drásticamente el espacio de búsqueda.

Backtracking con bitmasks

En cada fila 'y', se calculan las columnas libres (no atacadas) mediante tres bitmasks que acumulan las amenazas propagadas desde las filas superiores:

- Left: columnas amenazadas por diagonales que descienden hacia la izquierda.
- Down: columnas directamente ocupadas (ataque vertical).
- Right: columnas amenazadas por diagonales que descienden hacia la derecha.

Las columnas disponibles para colocar la reina se obtienen haciendo la operación OR entre estas 3 variables. Al avanzar a la siguiente fila, las amenazas se propagan desplazando los bitmasks: left se

desplaza un bit hacia la izquierda, right se desplaza un bit hacia la derecha y down permanece igual ya que es la misma columna.

Explotación de simetrías (función Check)

Un tablero de $N \times N$ tiene 8 transformaciones simétricas válidas: 4 rotaciones (0° , 90° , 180° , 270°) y sus respectivas reflexiones. Para evitar el cómputo redundante, el algoritmo está diseñado para hallar únicamente la solución con el menor peso, es decir donde la reina esté más cerca del origen (solución canónica) de cada grupo de simetría. La función Check() compara el tablero actual contra sus rotaciones fila a fila. Si alguna rotación tiene menor peso, descarta la solución actual (la canónica será encontrada en otra rama). Si ninguna rotación es menor, la solución actual es la canónica y se incrementa el contador correspondiente según su simetría:

- COUNT8: El tablero es completamente asimétrico (ninguna de sus 7 transformaciones coincide consigo mismo). Por ende, esta única solución representa 8 soluciones físicas distintas en el tablero real.
- COUNT4: El tablero posee simetría rotacional de 180° (es idéntico a su propia rotación de 180°). Esta solución canónica representa 4 soluciones en el espacio total.
- COUNT2: El tablero es altamente simétrico (es idéntico a su propia rotación de 90°). Representa únicamente 2 soluciones en el total.

División por casos en NQueens()

La función NQueens() divide estructuralmente la exploración del árbol de búsqueda en dos etapas diferenciadas para explotar las propiedades geométricas de simetría y exclusión de bordes:

- Primera Etapa (Backtrack1): Se ejecuta al fijar de forma estática la primera reina en la esquina superior izquierda del tablero ($\text{BOARD}[0] = 1$). Bajo esta condición, es matemáticamente imposible que ese tablero sea perfectamente simétrico si lo rotas. Además como el algoritmo explora de forma ordenada, sabe que ninguna otra rama generará un tablero que, al rotarse, tenga una reina en esta esquina específica de esta manera. Es por esto que toda solución hallada en esta etapa es una solución canónica asimétrica. Por lo tanto, al alcanzar la última fila, el algoritmo no invoca a la función Check() e incrementa directamente el contador COUNT8 (atribuyéndole el peso de 8 soluciones al total).
- Segunda Etapa (Backtrack2): Se dispara mediante un bucle que desplaza la primera reina a través de las columnas interiores (desde la columna inmediatamente contigua a la esquina hasta

alcanzar la mitad exacta del tablero, $N/2 - 1$). Dado que el lado derecho del tablero es un reflejo exacto del izquierdo, el algoritmo corta la exploración en la mitad, reduciendo el espacio de búsqueda al 50%. En esta etapa, cada solución hallada es analizada en la función `Check()` para filtrar aquellas configuraciones que no sean canónicas.

1.3 Almacenamiento de resultados y tableros válidos

Los resultados se acumulan en tres contadores globales (`COUNT8`, `COUNT4`, `COUNT2`). Al finalizar la búsqueda, en la función `NQueens()` se calcula:

$$\text{UNIQUE} = \text{COUNT8} + \text{COUNT4} + \text{COUNT2}$$

$$\text{TOTAL} = \text{COUNT8} \times 8 + \text{COUNT4} \times 4 + \text{COUNT2} \times 2$$

El contador `UNIQUE` representa las soluciones esencialmente distintas (sin contar rotaciones ni reflexiones) y el contador `TOTAL` representa todas las soluciones incluyendo las simétricas. Por último, el tablero `BOARD[N]` almacena la configuración de la última solución encontrada, pero no se conservan todas las soluciones en memoria simultáneamente.

2- Estrategia y descripción de las etapas de diseño paralelo

Para la transición hacia la arquitectura híbrida, se aplicó rigurosamente la metodología de diseño de cuatro pasos propuesta por Ian Foster y Grama.

2.1 Descomposición

Para el problema de las N reinas decidimos aplicar una descomposición híbrida. En principio utilizamos la descomposición por paralelismo de datos (en contraparte de la descomposición funcional) ya que el espacio de búsqueda se divide entre tareas que ejecutan el mismo código sobre distintos subconjuntos de datos. Todas las tareas realizan la misma operación sobre datos diferentes, sin dependencias entre sí durante la exploración. Dentro del paralelismo de datos se utilizó una estrategia exploratoria, propia de problemas de búsqueda combinatoria donde el espacio de soluciones se recorre dinámicamente. En este caso particular, a diferencia del esquema general de Branch and Bound, no es necesario calcular límites superiores e inferiores para podar ramas: la poda se reduce a descartar configuraciones con reinas en conflicto, ya que el objetivo es enumerar todas las soluciones válidas y no encontrar una óptima.

En el contexto de este problema, definimos una tarea como la exploración exhaustiva y autónoma de un subárbol de búsqueda de soluciones válidas, a partir de una configuración inicial fija de las dos

primeras filas (Fila 0 y Fila 1). En lugar de limitar las tareas al primer nivel del árbol (lo que generaría apenas $N/2$ tareas muy pesadas y desbalanceadas), expandimos la raíz de forma estática hasta el segundo nivel de profundidad. Al fijar simultáneamente las posiciones de la primera y la segunda reina, fragmentamos el espacio de estados en centenas de subárboles independientes. Esto nos permite obtener una granularidad más fina y por lo tanto mejor balance de carga.

En cuanto a las características de las tareas, las mismas son estáticas ya que se conoce de antemano todo el pool de tareas a ejecutar, sin embargo no se conoce la complejidad de las mismas, existiendo tareas extremadamente cortas, así como otras que requieren recorrer el árbol en su totalidad, por lo que las tareas tienen una naturaleza No uniforme. Por último, respecto a la dependencia entre tareas, todas ellas trabajan sobre conjuntos de datos diferentes, por lo que el problema se convierte prácticamente en 'embarrassingly parallel' teniendo como única dependencia a los contadores de soluciones, la cual se mitiga diseñando las tareas para que registren sus conteos de manera privada e independiente, delegando la consolidación total a una etapa de reducción posterior a la búsqueda.

2.2 Comunicación

La comunicación entre tareas de cómputo concurrentes es nula durante la fase de exploración ya que no existen dependencias de datos entre ellas mientras se recorren los subárboles de backtracking. La única fase de comunicación requerida se produce una vez que se han agotado todas las ramas de búsqueda, ejecutando un patrón de reducción global estructurada para acumular los contadores locales disjuntos de cada hilo y proceso en las variables globales consolidadas. Este patrón es estático, unidireccional, se produce una única vez y comunica un volumen mínimo de datos (tres enteros por proceso), por lo que su costo es igualmente despreciable.

2.3 Aglomeración

El objetivo fundamental de la etapa de aglomeración es evaluar si las tareas identificadas en la descomposición deben ser agrupadas en estructuras de mayor tamaño para reducir el overhead de control, o si deben preservarse de forma independiente para maximizar el paralelismo.

En nuestro diseño, se determinó no realizar una fusión o aglomeración secuencial de subtareas debido a que en la fase de descomposición refinamos la granularidad expandiendo el árbol hasta el segundo nivel de filas. Las tareas obtenidas ya poseen un tamaño óptimo de grano medio-grueso. Agruparlas de forma estática en bloques más grandes anularía el beneficio de contar con un pool abundante, agravando el problema del desbalance de carga intrínseco del algoritmo de Takaken.

No obstante, la aglomeración sí se aplicó de forma estricta sobre las estructuras de datos de control y de salida. Para minimizar el impacto en la transferencia y reserva de memoria, las variables de estado del backtracking se aglomeraron en una estructura compacta y liviana (struct Tarea). Debido a la eficiencia de los Bitboards de Takaken, el conjunto de datos necesario para describir un subárbol completo se reduce a unas pocas variables locales, logrando que el costo de manipulación de cada tarea sea muy bajo. Para evitar que los hilos compitan destructivamente por el acceso a variables globales de conteo (lo que provocaría condiciones de carrera o una degradación por *false sharing* en el caché del procesador), se decidió aglomerar los resultados de forma privada. Cada hilo acumula de forma autónoma en sus propios contadores locales disjuntos (count8, count4, count2).

2.4 Mapeo

Para la asignación física de las tareas al hardware subyacente, implementamos un esquema de mapeo híbrido estructurado en dos niveles jerárquicos independientes:

- Nivel de Memoria Distribuida (MPI - Mapeo Estático Cíclico):

Se instancian 2 procesos MPI (uno por cada nodo multicore físico). A nivel de red, el espacio total de tareas generadas se divide de forma estática utilizando una estrategia cíclica basada en el identificador único del proceso (rank). Debido a la naturaleza del problema, las ramas asociadas a los bordes del tablero son significativamente más cortas que las del centro. Si realizáramos una división por bloques contiguos (asignando la primera mitad de tareas al Nodo 0 y la segunda al Nodo 1), generaríamos un desbalance de carga crítico. Para evitarlo, aplicamos la operación módulo ($\text{tarea_id} \% \text{num_procs} == \text{rank}$), logrando que el nodo i tome las tareas i , $i+k$, $i+2k$, etc. (siendo k el número de nodos).

Este balanceo estadístico se ve fuertemente potenciado por la decisión previa (tomada en la etapa de descomposición) de expandir el árbol al segundo nivel. Al disponer de centenas de tareas en lugar de solo $N/2$, la distribución intercalada le asegura a cada nodo MPI una mezcla equitativa de subárboles externos e internos, promediando una carga de trabajo inicial prácticamente idéntica sin requerir comunicación por red.

- Nivel de Memoria Compartida (Pthreads - Mapeo Dinámico):

Dentro de cada nodo, el proceso MPI lanza un número parametrizado T de hilos de ejecución. Para mitigar la imprevisibilidad del tiempo de cómputo de cada subárbol, se descarta un reparto estático hacia los hilos y se implementa un mapeo dinámico mediante una Bolsa de Tareas (Bag of Tasks). Los hilos compiten activamente por las tareas de su pool local bajo demanda,

extrayéndolas una a una mediante un esquema protegido por variables de exclusión mutua. Este enfoque es significativamente más eficiente que el reparto estático entre hilos. Si un hilo recibe una tarea de procesamiento extenso, los demás continúan drenando el resto de la bolsa, logrando un balance de carga dinámico prácticamente perfecto con un overhead de sincronización mínimo.

3- Tiempos de ejecución, métricas y análisis de escalabilidad

En esta sección se presentan los resultados experimentales obtenidos al ejecutar el algoritmo secuencial y el paralelo en el cluster de la cátedra (2 nodos multicore), para cargas de trabajo $N \in \{14, 15, 16, 17, 18\}$ y configuraciones de 4, 8 y 16 unidades de procesamiento.

3.1 Tiempos de ejecución

En esta sección mostramos los resultados de ejecutar el algoritmo original de Takaken para los distintos valores de N . Por otro lado también mostramos los tiempos de ejecución de nuestro algoritmo paralelizado para cada combinación de N y número de UPs.

Tiempo[seg]	Número de reinas				
UPs	14	15	16	17	18
Secuencial	0.180	1.113	7.255	50.142	364.567
4	0.055	0.339	2.198	15.204	110.385
8	0.027	0.169	1.109	7.597	55.658
16	0.015	0.089	0.573	3.909	28.160

Tabla 1: tiempos de ejecución

3.3 SpeedUp absoluto

El Speedup absoluto toma como referencia el mejor algoritmo secuencial disponible (Algoritmo secuencial de Takaken proporcionado por la cátedra), lo que lo convierte en una métrica más exigente y representativa del beneficio real de la paralelización.

Se calcula de la siguiente forma: $S_{Abs}(P) = T_{mejor} / T_p(P)$

Donde T_{mejor} representa el tiempo para el mejor algoritmo secuencial y $T_p(P)$ el tiempo para P UPs

SpeedUp	Número de reinas				
UPs	14	15	16	17	18
4	3.002	3.092	3.060	3.090	3.069
8	6.049	6.084	6.054	6.155	6.119
16	10.247	11.351	11.570	11.877	11.898

Tabla 2: SpeedUp absoluto

Al observar las tablas vemos que para todas las cargas , agregar UPs implica una mejora significativa en el SpeedUp el cual se acerca cada vez más al ideal cuanto mayor sea la carga. Por lo tanto, si conviene utilizar un mayor número de UP para acelerar nuestro algoritmo. Por otro lado, al observar una columna de la tabla, para un problema de tamaño fijo, no observamos el límite propuesto por la ley de Amdahl, para encontrarlo se debería continuar agregando UPs hasta encontrar el punto en el cual se estanca el SpeedUp.

3.4 Eficiencia

La Eficiencia indica qué porcentaje del tiempo disponible las unidades de procesamiento realizan trabajo útil. Se calcula como: $E(P) = S(P) / P$

Eficiencia	Número de reinas				
UPs	14	15	16	17	18
4	0.751	0.773	0.765	0.772	0.767
8	0.756	0.761	0.757	0.769	0.765
16	0.640	0.710	0.723	0.742	0.744

Tabla 3: Eficiencia absoluta

A partir de estas definiciones y de los resultados obtenidos podemos afirmar que nuestro programa es débilmente escalable ya que si observamos las diagonales de la tabla donde se aumenta simultáneamente las UPs y la carga, la eficiencia se mantiene aproximadamente constante. Por otro lado también podemos afirmar que no es fuertemente escalable bajo los parámetros analizados (4, 8 y 16 UP) ya que la eficiencia se degrada a medida que se agregan UPs.

3.5 Balance de carga

El balance de carga se calcula como $B = \text{Promedio}(T)/\text{Maximo}(T)$. Los valores obtenidos oscilan entre 0.909 y 0.992, lo que indica que el pool dinámico de tareas distribuye la carga de manera muy equitativa. Para valores mayores de N, donde las tareas son más numerosas y el cómputo por tarea es más homogéneo, el balance mejora consistentemente.

Problema [N]	UPs	Tmax [s]	Promedio Total [s]	Balance
14	4	0.059040	0.058	0.978
	8	0.029573	0.029	0.976
	16	0.016462	0.015	0.909

15	4	0.359942	0.357	0.991
	8	0.182073	0.179	0.983
	16	0.096931	0.091	0.942
16	4	2.370674	2.324	0.980
	8	1.197192	1.172	0.979
	16	0.626685	0.593	0.947
17	4	16.22744	16.095	0.992
	8	8.145908	8.067	0.990
	16	4.221601	4.094	0.970
18	4	118.8052	116.803	0.983
	8	59.58104	58.556	0.982
	16	30.63996	29.713	0.970

Tabla 4: Balance de carga

3.6 Análisis de escalabilidad

El análisis de escalabilidad tiene como objetivo estudiar cómo se comporta el algoritmo al incrementar la capacidad de cómputo, ya sea aumentando las unidades de procesamiento, el tamaño del problema, o ambos simultáneamente. Se distinguen dos tipos de escalabilidad:

- Escalabilidad fuerte: la eficiencia se mantiene aproximadamente constante al incrementar el número de UPs con carga de trabajo fija. Se analiza observando columnas de la tabla de eficiencia.

- Escalabilidad débil: la eficiencia se mantiene aproximadamente constante al incrementar simultáneamente UPs y tamaño del problema. Se analiza observando las diagonales de la tabla de eficiencia.

El análisis de escalabilidad se realiza a partir de la tabla de eficiencia absoluta, observando columnas para escalabilidad fuerte y diagonales para escalabilidad débil, siguiendo la metodología del apunte.

Escalabilidad fuerte

Para cada N fijo se analiza si la eficiencia se mantiene al aumentar las UPs de 4 a 8 a 16. Para N=14, la caída es pronunciada: de 0.751 a 0.640. Este comportamiento es consistente con la ley de Amdahl: para una carga fija pequeña, la fracción secuencial representa una porción mayor del tiempo total y limita más el Speedup al aumentar P. Se concluye que el algoritmo no es fuertemente escalable ya que la eficiencia no se mantiene constante al aumentar las UPs.

Escalabilidad débil

Se analiza la diagonal de la tabla de eficiencia correspondiente a incrementar simultáneamente N y las UPs. Tomando la diagonal (4 UP, N=14) $\rightarrow$ (8 UP, N=16) $\rightarrow$ (16 UP, N=18), los valores de eficiencia absoluta son 0.751, 0.757 y 0.744. La eficiencia se mantiene prácticamente constante alrededor del 74-75%, demostrando que el algoritmo es débilmente escalable en el rango medido. Esto es coherente con la ley de Gustafson-Barsis: al aumentar la carga de trabajo proporcionalmente a los recursos, la fracción secuencial pierde peso relativo y el Speedup crece aproximadamente lineal con P.

4- Uso de IA

Se utilizaron herramientas de inteligencia artificial como asistentes en un proceso de co-creación para el análisis del algoritmo secuencial, la validación de estrategias de diseño paralelo y el desarrollo de la solución.

Herramienta: Gemini		
Prompt	Éxito	Observación
Genera una estructura base en C para paralelizar el problema de las N-Reinas utilizando un enfoque híbrido con MPI y Pthreads.	NO	Estructura de archivos confusa y código innecesario.
Clasifica el tipo de descomposición aplicada en un algoritmo de N-Reinas que expande estáticamente los primeros niveles del árbol de búsqueda. ¿Se trata de una descomposición de datos que contiene una estrategia exploratoria, o viceversa?	SI	La respuesta permitió clarificar la jerarquía teórica de Grama y Foster. La categoría principal es el paralelismo de datos, adoptando una especialización exploratoria.
¿Consideras que es conveniente fijar las reinas de las primeras 2 filas del tablero para obtener mayor cantidad de tareas y lograr un mejor balance de carga?	SI	Respondió que sí es conveniente.

Tabla 5: prompts gemini

Herramienta: Claude		
Explicó el funcionamiento del algoritmo secuencial nreinas.c línea por línea, detallando la lógica de backtracking, el uso de bitmasks y el mecanismo de explotación de simetrías mediante la función Check().	SI	Generó un explorador interactivo con explicaciones por sección y por línea del código secuencial nreinas.
¿Consideras que usar una estrategia de descomposición por paralelismo de datos en conjunto con una exploratoria es un buen planteo para paralelizar el algoritmo nreinas.c con una arquitectura híbrida MPI+Pthreads sobre un cluster de 2 nodos multicore?	SI	Consideró que si.
Comparó el mapeo estático con partición cíclica y el mapeo dinámico con pool de tareas para el problema de N-Reinas, evaluando cuál es más adecuado dado el desbalance de carga entre tareas.	SI	Concluyó que el pool dinámico es superior para este problema dado el desbalance pronunciado entre subárboles de distintas columnas.

Generará una estructura modular de tres archivos para el algoritmo paralelo de N-Reinas: MPI_source.c con la lógica de distribución entre nodos, Pthreads_source.c con la creacion y funcionamiento de los hilos y Pthreads_source.h con las estructuras y prototipos compartidos.	Parcial	Generó correctamente la estructura general de los 3 archivos pero ubicó algunas funciones en archivos erróneos.
Agregá comentarios detallados línea a línea en las funciones Backtrack1(), Backtrack2(), Check() y generar_tareas(), explicando el propósito de cada operación de bits, las restricciones de simetría y la lógica de propagación de amenazas.	SI	Se documentaron todas las funciones con comentarios.

Tabla 6: Prompts claude